

Algorithmique avancée – Examen Réparti 2
UPMC — Master d’Informatique —
Janvier 2016 – durée 2h

*Les seuls documents autorisés sont les polys de cours, ainsi que la copie double personnelle.
Le barème donné est indicatif.*

1 QCM [6 points]

Cocher une seule réponse par question, et argumentez votre choix.

Q1. La récurrence $T(n) = T(n/3) + \Theta(1)$ a pour solution

- A) $T(n) = \Theta(n)$
- B) $T(n) = \Theta(n \log n)$
- C) $T(n) = \Theta(\log n)$

Donner un exemple d’algorithme qui a cette complexité.

Q2. En utilisant un splay-tree de n clés, on peut réaliser un tri des n clés

- A) en temps moyen $O(n \log n)$
- B) en temps amorti $O(n \log n)$
- C) en temps pire $O(n \log n)$

Q3. Quel algorithme de compression statistique nécessite de trier les fréquences en début de traitement ?)

- A) Huffman statique
- B) Huffman dynamique
- C) Shannon-Fano

Rappeler le principe de cet algorithme.

Q4. Soit $T = (ab)^{1000}$ le texte constitué de la répétition de mille fois ab . Pour compresser T , la meilleure méthode est

- A) Huffman statique
- B) LZW
- C) Shannon-Fano

Q5. On représente un code préfixe complet par

- A) les feuilles d’un trie binaire
- B) les noeuds internes d’un trie binaire
- C) les feuilles et les noeuds internes d’un trie binaire

Q6. Dans la compression JPEG, l’image est découpée en blocs de 8×8 pixels. Si l’on découpe en blocs de 16×16 , le nombre de multiplications effectuées lors de la transformée cosinus discrète matricielle est

- A) divisé par 2
- B) multiplié par 2
- C) multiplié par 4

2 Exercice [4 points]

a) On considère un texte formé de 5 lettres a, b, c, d, e , de fréquences respectives 1, 1, 1, 2, 3. Donner les différents arbres de Huffman construits sur ce texte. Quelle est le poids de ces arbres ($\text{poids}(H) = \sum f(\nu) \text{prof}(\nu)$; ν feuille de H) ?

Que signifie ce poids au niveau de la compression ?

b) On considère la suite (g_k) définie par $g_0 = g_1 = g_2 = 1$ et pour $k \geq 2$, $g_k = g_{k-1} + g_{k-2}$. Le texte T contient $k + 1$ lettres qui ont pour fréquences $g_0 \dots g_k$.

Quel est le poids d’un arbre de Huffman pour T (votre réponse doit être accompagnée d’une preuve) ?

Ecrire un algorithme qui construit un arbre de Huffman de hauteur k pour T .

3 Problème [10 points]

On considère ici une méthode de compression qui utilise le fait que certains mots sont absents du texte à compresser. On se place dans un alphabet binaire $\{0, 1\}$. Le principe de la compression est le suivant : si on sait par exemple que le

facteur 1011 n'apparaît pas dans le texte, cela veut dire qu'à chaque fois que le facteur 101 est présent, il sera suivi par 0, et l'on peut donc omettre ce 0.

Soit un texte T , on note $F(T) = \{w \in \{0,1\}^*; \exists A, B \in \{0,1\}^* \text{ avec } T = AwB\}$ l'ensemble des *facteurs* de T . Un ensemble de *non-facteurs* de T , $NF(T)$ est un ensemble de mots qui ne sont pas dans T , c'est-à-dire un sous-ensemble (fini ou non) du complémentaire de $F(T)$. Cet ensemble peut être donné en extension ou décrit par une propriété. Par exemple pour $T = 0101$, on peut avoir $NF(T) = \{00; 11\}$ ou alors $NF(T)$ = l'ensemble des mots contenant deux lettres 1 à la suite.

Question 1. Donner deux ensembles $NF(T)$ différents pour $T = 100101$.

On suppose que l'ensemble $NF(T)$ des non-facteurs du texte T est fourni ; on compresse T dans un parcours de gauche à droite, en éliminant toutes les lettres pouvant être déduites des lettres précédentes à l'aide de $NF(T)$: supposons que l'on a déjà compressé le préfixe v de T ($T = vv'$). S'il existe dans $NF(T)$ un mot $u = u'x$, $x \in \{0,1\}$, tel que u' est un suffixe de v , alors la lettre suivant v est nécessairement $\bar{x}$ (si x vaut 0 alors $\bar{x}$ vaut 1, et inversement). Cette lettre est inutile dans le texte compressé car elle peut être déduite grâce à $NF(T)$. On élimine de cette manière toutes les lettres pouvant être déduites des lettres précédentes à l'aide de $NF(T)$.

Par exemple la compression du texte $T = 01101010$ avec $NF(T) = \{00; 111; 1011\}$ aboutit au texte compressé $TC = 01$, après avoir traité la suite des préfixes de T , comme montré ci-dessous .

préfixe v	TC	mot dans $NF(T)$
0	0	00
01	0	—
011	01	111
0110	01	00
01101	01	1011
011010	01	00
0110101	01	1011
01101010	01	

Question 2. Appliquer la méthode à la compression du texte $T = 01101010$ avec $NF(T) = \{00; 0111\}$. Faire le même type de tableau que ci-dessus.

Question 3. Décrire, en pseudo-code, un algorithme de compression suivant la méthode précédente. Vous explicitez les primitives que vous utilisez en donnant leur spécification. Quelle est la complexité de cet algorithme ?

Inversement, pour décompresser un texte TC à partir de l'ensemble de non facteurs $NF(T)$ qui a été utilisé lors de la compression, on construit la suite des préfixes du texte initial T : à partir de la première lettre de TC , on retrouve, à l'aide de $NF(T)$, les lettres qui suivent dans T ; quand on ne peut plus deviner la lettre suivante de T , on examine la prochaine lettre dans le texte compressé TC et l'on applique le même processus ; et ainsi de suite jusqu'à épuisement des lettres de TC .

Par exemple pour décompresser $TC = 01$ à l'aide de $NF(T) = \{00; 111; 1011\}$, on procède comme suit :

préfixe v	TC	mot dans $NF(T)$
0	01	
01	1	00
011	1	on ajoute le dernier bit
0110	ε	111
01101	ε	00
011010	ε	1011
0110101	ε	00
01101010	ε	1011

Question 4. Appliquer l'algorithme pour décompresser le texte TC obtenu en question 2, à l'aide de $NF(T) = \{00; 0111\}$.

Question 5. Expliquer pourquoi dans l'algorithme de décompression il faut aussi fournir la *taille* du texte initial T .

Question 6. Décrire, en pseudo-code, un algorithme de décompression suivant cette méthode, et donner sa complexité.

Question 7. Il s'agit maintenant de trouver une structure de données pour représenter l'ensemble $NF(T)$. Quelle est l'opération sur laquelle cette structure de données doit être efficace.
Proposer une représentation de l'ensemble $NF(T)$.